

PLEASE LEAVE ANY COMMENTS TO IMPROVE THE ASSIGNMENT

IF YOU FIND BETTER EXERCISE ON INTERNET USE THAT & LET US KNOW

[SkillTrack: Developer Learning Management System](#)

[Summary](#)

[Learning Objectives](#)

[Project Modules](#)

[Environment Setup](#)

[Core Acceptance Criteria](#)

[1. User Management](#)

[2. Course & Enrollment](#)

[3. Spring Batch Job](#)

[4. Quartz Scheduler](#)

[5. Caching](#)

[6. Auditing & Notifications](#)

[7. Metrics & Health](#)

[8. Testing](#)

[Developer Deliverables](#)

SkillTrack: Learning Management System

Summary

Build a **modular Learning Management System (LMS)** where developers can register, enroll in courses, complete coding challenges, and track their progress.

The focus is on **enterprise-grade backend design**, leveraging advanced Java and Spring capabilities (persistence, scheduling, security, caching, testing, etc.).

The project should be fully executable on a **developer's local machine** using **Docker**, **H2/PostgreSQL (free)**, and **free cloud APIs (like SendGrid free tier, GitHub OAuth, etc.)**.

Learning Objectives

By the end of this project, engineers should demonstrate mastery of:

1. Advanced **JPA/Hibernate** concepts: custom queries, entity graphs, caching, auditing.
 2. **Spring Security**: method-level security, OAuth2 login, JWT, and role hierarchies.
 3. **Spring Batch**: batch job orchestration for data processing.
 4. **Quartz Scheduler**: scheduled background jobs with persistence.
 5. **Spring Caching & Transactions**: managing performance and consistency.
 6. **Spring Profiles, Actuator, and Metrics** for runtime observability.
 7. **Testing**: integration testing with Testcontainers, MockMvc, and embedded DBs.
 8. **API Documentation** with SpringDoc (OpenAPI 3).
-

Project Modules

Module	Description	Core Technologies
1. User Management	Registration, login, and profile management. Includes OAuth2 login with GitHub.	Spring Security, JWT, OAuth2 Client, Spring Data JPA
2. Course & Enrollment	CRUD for courses and modules, enrollment flow, and progress tracking.	Spring Data JPA, DTO mapping (MapStruct), Validation API
3. Batch Job Processing	Periodic job to import course content or archive old user data.	Spring Batch, H2/PostgreSQL
4. Scheduling	Automatic reminder emails to users about incomplete courses.	Quartz Scheduler, Spring Email (JavaMailSender)
5. Reporting & Metrics	API endpoints for activity reports and performance monitoring.	Spring Actuator, Micrometer, Prometheus/Grafana (optional)
6. Security & Access Control	Role-based access for Admin, Instructor, Student.	Spring Security with role hierarchy and method-level security

7. Caching & Performance	Cache course details and user stats with expiration.	Spring Cache with Caffeine or Redis
8. Audit & Notifications	Entity change auditing and notification triggers.	Hibernate Envers, Async events, SendGrid free API

Environment Setup

Stack Requirements:

- Java 21
- Spring Boot 3.x
- Maven or Gradle
- PostgreSQL (Dockerized)
- Redis (Dockerized, optional)
- Testcontainers for integration testing
- Docker Compose for local orchestration

Repository Structure Example:

```
skilltrack-lms/
├─ skilltrack-api/           # Spring Boot app (controllers,
services, config)
├─ skilltrack-batch/         # Spring Batch jobs
├─ skilltrack-scheduler/     # Quartz jobs
├─ skilltrack-common/        # Shared entities, utils, and DTOs
├─ docker-compose.yml
└─ README.md
```

Core Acceptance Criteria

1. User Management

- Users can register with email/password or login via GitHub OAuth.
- JWT tokens are issued for authenticated users.
- Admin users can manage roles.
- Passwords must be hashed with BCrypt.
- Session management should be stateless.
- Sensitive endpoints (like `/admin/**`) must be role-protected.
- User details are stored in PostgreSQL using JPA.

2. Course & Enrollment

- Instructors can create and update courses.
- Students can enroll/un-enroll from available courses.
- Course entity must have a one-to-many relationship with modules.
- Lazy vs. eager loading should be optimized using entity graphs.
- Include soft deletes (logical deletion flag).

3. Spring Batch Job

- A scheduled batch job imports course metadata from a CSV file.
- Batch steps: read (CSV) → process (validate) → write (DB).
- Failed records must be logged to a separate table.
- The job can be manually triggered via an API.

4. Quartz Scheduler

- Quartz triggers a daily email reminder for users with incomplete courses.

- Use persistent job store (database-backed).
- Job history should be queryable via REST API.

5. Caching

- Cache course details and instructor profiles with a configurable TTL.
- Implement cache invalidation when entities are updated.
- Support toggling caching via Spring Profiles.

6. Auditing & Notifications

- All entity changes (create/update/delete) should be tracked using Hibernate Envers.
- A post-update event should trigger a notification (async) to SendGrid free API.

7. Metrics & Health

- Expose `/actuator` endpoints for health, metrics, and scheduled job status.
- Include custom metric: number of completed enrollments per day.

8. Testing

- Use JUnit 5 and Testcontainers for PostgreSQL/Redis.
 - Integration tests must cover:
 - Authentication flow
 - Course CRUD operations
 - Batch job execution
 - Quartz scheduling triggers
-



Developer Deliverables

Each participant/team must deliver:

1. A runnable Spring Boot application (`./mvnw spring-boot:run`).
2. Docker Compose file for database and Redis.
3. Test reports (JUnit) and code coverage summary.
4. README including:
 - Setup and run instructions
 - API endpoints documentation (via OpenAPI 3)
 - Explanation of each module
 - Technologies used